

An Agentic Question-Answering System for Cross-Office Administrative Procedure Integration

Yao-Ming Pai

Student ID: 112703056

Instructor: Prof. Ming-Feng Tsai

Department of Computer Science, National Chengchi University

Code: <https://github.com/AlanPai777/nccu-academic-rag>

Contents

Summary

1. Problem

- 1.1 The task
- 1.2 Why I did not use a standard RAG pipeline
- 1.3 Why local deployment
- 1.4 The problem this creates
- 1.5 Approach and contributions

2. Terms used in this report

3. System design

- 3.1 Retrieval: keyword search and whole pages
- 3.2 The loop
- 3.3 Routing by code
- 3.4 Compound queries
- 3.5 Synthesis and self-evaluation
- 3.6 Example: one query from start to finish

4. Data

5. Evaluation setup

- 5.1 Queries
- 5.2 Scoring
- 5.3 How the scoring can go wrong
- 5.4 Size of the evaluation

6. Results

- 6.1 Basic queries
- 6.2 Cross-office patterns
- 6.3 Compound query

7. What I changed and what happened

- 7.1 Four changes
- 7.2 The cross-institution failure

8. Limitations

9. Related work

10. Conclusion

References

Summary

At a university, the steps for one administrative task are spread across many offices' web pages and PDF forms. To answer "how do I apply for a leave of absence?", a system has to find the procedure, find the form, and list every office that must sign it, with each office's location and phone number.

I built a question-answering system for this task and ran it on public regulations from National Chengchi University (NCCU). It runs on Gemma4:31b (Gemma Team, 2026), an open-weight model that can be deployed locally, with no fine-tuning. It does not use vector search. It searches the raw text with keywords and reads whole pages, and the language model decides which tool to call next. A small open-weight model is unreliable at choosing tools on its own, so I draw on and integrate five established techniques into a single multi-turn architecture: chain-of-thought prompting (Wei et al., 2022), ReAct (Yao et al., 2023), Plan-and-Solve (Wang et al., 2023), query routing (Jeong et al., 2024), and self-evaluation (Asai et al., 2024). Where a decision can be checked by code, such as a keyword rule or a pattern match, I make it in code and use the model only as a fallback.

On 11 test queries, scored against hand-checked checklists, the system gets most required content right. Some queries fail clearly. A form-download question got a full procedure explanation instead of a short answer with a link, and a question about a multi-university program first got 3 of 14, then 9 of 14 after I fixed a bug in the relevance judge (Section 7).

1. Problem

1.1 The task

Take the query 如何辦理休學 ("how do I apply for a leave of absence"). The official form is QP-T01-03-02. It has a signature chain of nine stops across seven offices. Some stops apply only under certain conditions, and the chain ends with three levels of approval up to the dean's office. A good answer names every stop, says which office each one is, gives that office's contact details, and links the form.

The form (December 2025 edition) lists the stops in this order:

Stop	Office	Where and who
1	Department (系所)	The student's own department office
2	Library (圖書館)	Circulation desk of the main library or a branch
3	Cashier's office (出納組)	Administration building, 5th floor. Confirms the student account
4	Housing office (住宿組)	3rd floor. Dorm residents only
5	Overseas Chinese and Foreign Student Affairs (生僑組) or Indigenous Peoples Resource Center (原資中心)	3rd floor. Overseas Chinese, Hong Kong, Macau and mainland students go to 生僑組. Indigenous students go to 原資中心
6	International Cooperation Office (國際合作事務處)	8th floor. International students only

Stop	Office	Where and who
7	Registrar's office (教務處註冊組): handler	4th floor. Proposes the decision
8	Registrar's office: division head (組長)	Same office
9	Registrar's office: dean of academic affairs (教務長)	Final approval

That is nine stops in seven offices. Stops 4 and 6 apply only to some students.

The same holds for the other queries in this report: a department transfer, a dorm-deposit refund, a double major, exchange-credit exemption. Each needs facts from several pages and several offices.

1.2 Why I did not use a standard RAG pipeline

My first system was a conventional one: documents split into chunks, embedded with `embeddinggemma`, stored in Qdrant, merged with an FTS5 keyword index, reranked, and answered by a local model (`qwen2.5:7b`). I saw three problems.

- **Speed.** With a cross-encoder reranker running on CPU, one query (如何辦理休學) took over ten minutes. The reranking step alone took 9 to 10 minutes. Replacing it with reciprocal rank fusion (RRF) cut the time to about 100 seconds, but that change did not touch the two problems below.
- **Answer shape.** The answers tended to reproduce regulation text. The answer to the leave-of-absence query described the regulation and did not list the signature fields the form requires. The pipeline could not follow a regulation page to the form it mentions, and then to each office on the form (a multi-hop problem).
- **Index cost.** Building the dense index for one subdomain (`aca`, about 14,430 chunks) took roughly 40 to 60 minutes. Cost grows with the corpus.

Chunks are also a poor fit for this task. The facts needed for one answer sit on different pages, and no single chunk contains them together.

1.3 Why local deployment

Documents like these are often internal, and an institution may not be allowed to send them to a closed-source API. I therefore want the system to work on a model that can run locally. I test it here on public regulations, which lets anyone reproduce the results. Whether it would work on confidential documents is not tested.

1.4 The problem this creates

If the model chooses its own tools, a small open-weight model is unreliable at it. In my first agentic version, the model would often find the right regulation page and then never open the form the page mentioned. The answer cited the correct regulation but left out the signature chain, because the model never called the tool that reads the form. Zhou et al. (2026) report a related result: when a model was left to choose its own reasoning strategy with no training, this worked only for GPT-5 and failed for weaker models. This is the problem the rest of the report deals with.

1.5 Approach and contributions

The approach has three parts. First, the system uses no embeddings. It searches with keywords, reads whole pages, and lets the model choose each tool call. Second, to keep tool calling stable on a small open-weight model, I draw on and combine five established techniques into a single multi-turn architecture (Section 3). Third, wherever a decision can be checked by code, I check it in code and use the model only as a fallback.

The contributions are:

1. **Retrieval design.** Chunk-based retrieval is a poor fit for procedures that need information from several offices and forms. I replace it with agentic retrieval based on keyword search and whole-page reading.
2. **Architecture.** I integrate chain-of-thought prompting, ReAct, Plan-and-Solve, query routing, and self-evaluation into a single architecture for stabilizing multi-turn tool calling. It runs on Gemma4:31b with no fine-tuning. The individual techniques are not new. The contribution is integrating them for this task. I did not test each one separately.
3. **Cross-office evaluation.** I built and hand-checked five extra queries. Each isolates a different coordination pattern: parallel approval, sequential handoff, a three-stage chain, conditional routing, and cross-institution routing. Reliability differs by pattern. The system reached full marks on the parallel and sequential cases and did worst on the cross-institution case (3 of 14 at first). I traced that failure to a bug in the relevance judge and fixed it with a general data-level change, not a rule written for that one query.
4. **An observation.** In four cases (form fetching, office-name detection, candidate filtering, and link stripping), replacing a model decision with a code check, removing it, or cleaning its input in code improved the results. I saw this inside one system. I did not test it as a general rule.

2. Terms used in this report

Term	Meaning here
Stop	One signature slot on a form, filled by one office.
Subdomain	One of 153 NCCU web sites (for example <code>aca</code> , the academic affairs office). Tools can be limited to one subdomain.
Tool	A function the model can call. There are five (Section 3.1).
Turn	One round of the agent loop: restate the goal, call a tool, read the result.
Resource lookup	A step that fetches a form by its ID and puts the form text into the conversation.
Contact lookup	A step that looks up an office's phone, extension, and location in a contact index.
Synthesis	The final model call that writes the answer from everything gathered.
Self-evaluation	A check on the draft answer. If it fails, the system goes back for another attempt.
Checklist	The hand-built list of required content used to score an answer (Section 5).

3. System design

This section follows the order in which the design grew. Each part answers a problem that the previous part left open.

The design combines established techniques. The table shows which technique each part of the system uses. None of them requires training a model.

Technique	Source	Used for	Section
Chain-of-thought prompting	Wei et al., 2022	The model states its current goal in words before each tool call	3.2
ReAct (interleaved reasoning and acting)	Yao et al., 2023	The loop: restate the goal, call a tool, read the result, repeat	3.2
Plan-and-Solve	Wang et al., 2023	A planning pass before any tool call: classify the query and pick the subdomain	3.2, 3.3
Query routing	Jeong et al. (Adaptive-RAG), 2024	A two-layer classifier that sends each query to one of four routes	3.3
Query decomposition	Chan et al., 2024; Ma et al., 2023	Splitting a compound query into sub-queries	3.4
Self-evaluation	Asai et al. (Self-RAG), 2024	A two-stage check on the draft answer	3.5
Whole-unit reading, no embeddings	Jiang et al. (LongRAG), 2024; Wang et al. (ELITE), 2025	Reading whole pages, with no dense index	3.1

Two of these methods need training. Adaptive-RAG trains a classifier, and Self-RAG fine-tunes the model. My router uses keyword rules with a model fallback, and my self-evaluation uses a code checklist with a model check after it, so neither needs training data. I did not compare accuracy against those methods, and I did not run ablations, so I cannot say how much each technique contributes.

Figure 1 shows the control flow.

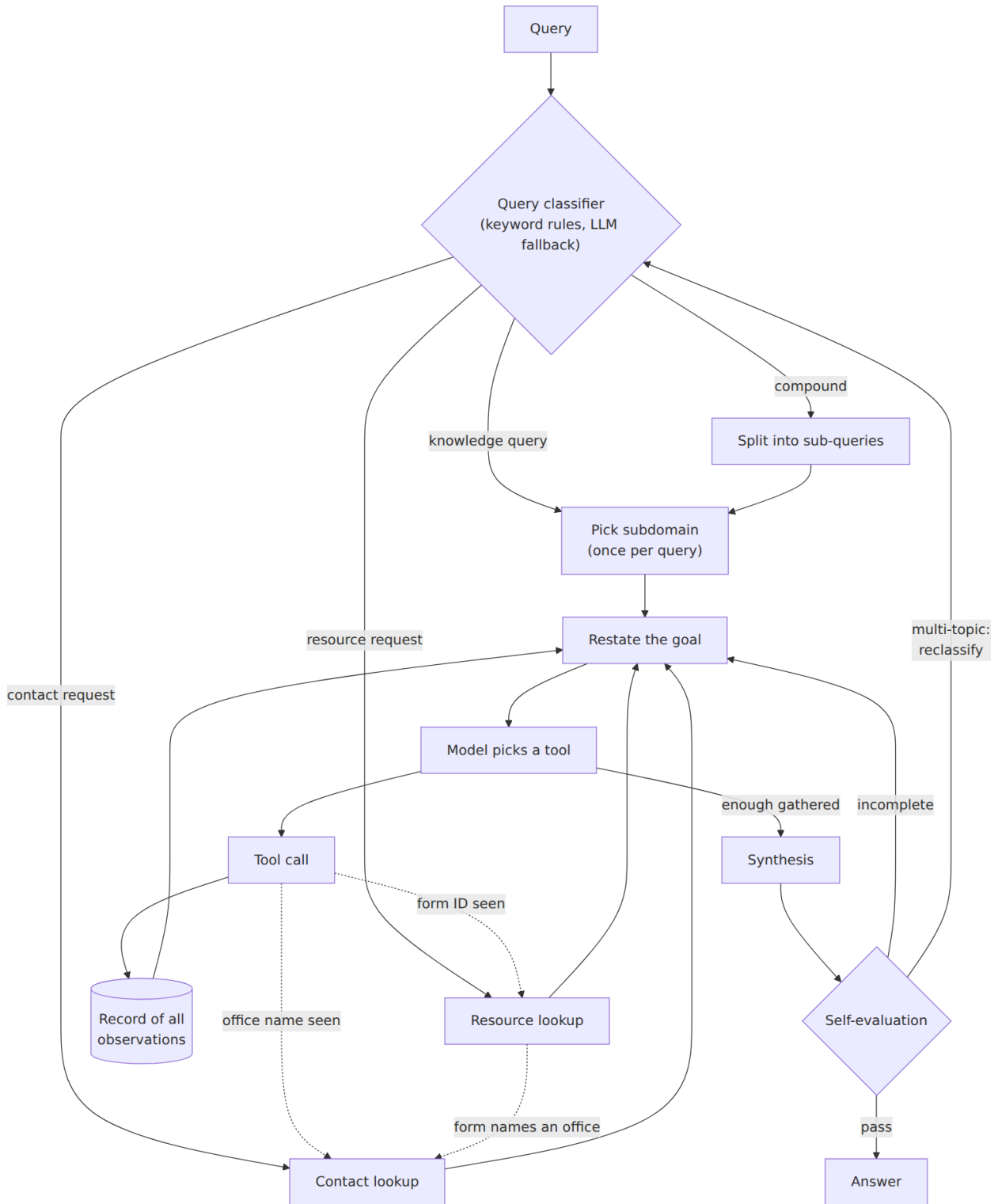


Figure 1. Control flow. Solid arrows are the normal path. Dashed arrows are the two automatic redirects described in Section 3.3. A rejected answer goes back to the loop if it is incomplete, or back to the classifier if the query had several topics (Section 3.5).

3.1 Retrieval: keyword search and whole pages

A query that needs several offices and forms fits chunk retrieval badly. The facts it needs are not in any one passage. A single similarity lookup cannot know in advance which fragments, out of many spread over unrelated pages, belong together. So I did not embed the corpus.

The corpus is indexed only for full-text search, with SQLite FTS5. Candidates are ranked with BM25 (Robertson et al., 1994). The model gets five tools and calls them as often as it judges necessary:

1. `search_texts`: keyword search. It returns the top five candidates, and a short model call decides whether any is relevant and picks the best.
2. `grep_texts`: literal substring search, used for exact strings such as form IDs.
3. `get_page`: read one page in full.
4. `extract_links`: list the hyperlinks on a page.
5. `get_form`: fetch a form by its ID, using an index that maps each form ID to its download URL and metadata.

Each tool returns whole pages or documents, never fragments. The tools search the chosen subdomain first and fall back to all subdomains if nothing is found there.

Everything a tool returns stays in the conversation until the answer is written, so a page read several turns ago is still available at synthesis time. This is a second route to the goal of LongRAG (Jiang et al., 2024): give the model more raw, unfragmented context. LongRAG retrieves one long unit at once. Here the model accumulates several whole pages over one query's tool calls. Subramanian et al. (2026) report that agentic keyword search reaches over 90% of the performance of conventional RAG.

Over a long conversation this record would grow without bound, and pages fetched for an earlier question could leak into a later answer. So between questions in the same conversation, I shrink the previous turn to just the question and its answer.

The model decides which tool to call, how many times, and in what order. This freedom does not by itself give reliable multi-step behavior. Section 3.2 describes how I stabilize it.

3.2 The loop

With five tools and no fixed order, a model can fail to call a tool the query needs, even though the tool exists and the information can be retrieved. In my first agentic version it found the right regulation page and never opened the form the page mentions. The loop below structures the model's freedom without removing it. Each turn:

1. **Restate the goal.** Before any tool call, the model writes what it is now looking for, given everything read so far. This is chain-of-thought prompting (Wei et al., 2022). It keeps the current goal as a persistent piece of state that I can inspect, instead of leaving it implicit inside one long generation.
2. **Pick the subdomain (turn 1 only).** The query is routed to one subdomain before any tool call, which narrows what the tools search. This is an upfront planning pass in the spirit of Plan-and-Solve (Wang et al., 2023). It runs once and is not interleaved with the later steps.
3. **Call a tool and read the result,** then go back to step 1. Interleaving a stated goal, an action, and the observation that follows is the ReAct pattern (Yao et al., 2023). It is what lets the second call, reading

the form, actually get made. Each step depends on what the previous one returned, and nothing is planned blindly before anything is retrieved.

The model decides when it has gathered enough to stop. There is no fixed number of turns.

Even inside this loop, some decisions come back in every query in a narrow, checkable form: whether the page just read names a form, or names an office. Leaving these to the same open-ended judgment that governs the rest of the loop brings back the problem this section set out to solve. Section 3.3 describes how I settle them in code.

3.3 Routing by code

Two routing decisions are made by fixed rules instead of the model's own choice of tool. They are the two shortcuts out of the loop in Figure 1.

Classifying the query. Before any tool call, the query is put into one of four classes: compound, resource request, contact request, or knowledge. A procedure-style question such as 如何辦理休學 is labeled a procedure, but it takes the same path as a knowledge query. Keyword rules are tried first. Only if no rule gives a clear result (none matches, or two classes tie) does a model call classify it. A resource or contact request goes straight to its lookup step and skips the loop's first pass. These queries have a narrow, recognizable shape ("where do I download form X", "what is office Y's extension") that the open loop would otherwise have to rediscover by trial and error each time.

Watching tool output. While the loop runs, every tool result is also checked with two fixed patterns, one for a form ID and one for an office name, whatever the model was doing at that step. A match sends the system to the resource lookup or the contact lookup. The model cannot decline this. I adopted it after seeing, in several cases, that a code check was more reliable than the equivalent model judgment whenever an objective signal existed (Section 7).

What the two lookups do.

- The **resource lookup** fetches every form whose ID was detected, plus one extra pass for forms that a fetched form itself references. It does not ask the model which forms are relevant. A wrong filter here would lose content with no way to recover it, so relevance is left to synthesis (Section 7.1). It also reads the form's signature table and records which stops have several approval layers, such as a division head and then a dean. The self-evaluation uses this (Section 3.5).
- The **contact lookup** scans the text for office names: a substring scan over about 433 known office names, combined with a model judgment. It fetches the contact details of every office it finds, again with no relevance filter, because every stop on a form is required. The details come from a contact index built from the offices' own web pages.
- A fetched form often names the office where it must be submitted. So the resource lookup also checks its own result for an office name and, if it finds one, runs the contact lookup next.

Both lookups then return to the loop, so the model can decide whether more is needed. A query resolved entirely by these steps still gets one pass through the loop.

3.4 Compound queries

A message like 如何辦理休學，圖書館的電話是多少 asks two unrelated things. It fits none of the routes above. The evidence for one half has nothing to do with the other half, and a single loop that handles both risks leaking pages from one question into the other's answer. So the message is split into independent sub-queries. A query counts as compound when it splits, at commas and full stops, into clauses that the keyword rules assign to at least two different classes. Each sub-query runs through its own loop, one after another, and reaches its own synthesis. The answers are joined at the end.

Related ideas are in RQ-RAG (Chan et al., 2024), which decomposes a query before searching, and in query rewriting (Ma et al., 2023). My split is a fixed rule based on punctuation, not a learned rewriting step. It works for the narrow case it targets: two administrative questions joined in one message. It misses compound queries whose parts fall in the same class, such as a question comparing two procedures. Only the self-evaluation can catch those, after the fact (Section 3.5). It is not built for the open-ended reformulation that the cited methods handle.

3.5 Synthesis and self-evaluation

Once the loop, a lookup, or the compound split has gathered what it judges to be enough, a synthesis step writes the answer from everything read. Correct routing and successful tool calls do not by themselves guarantee a complete answer. Content that the corpus contained can still be dropped in synthesis. So a self-evaluation runs after every path through the system, including the paths that went straight to a lookup.

It has two stages:

- **Stage 1 is a checklist in code**, with no model call. It is deliberately short and holds only checks that rarely give false alarms. For a procedure-style query, the answer must use a numbered-step format. If the form's signature table shows a stop with several approval layers, the answer must reflect at least one of those layers.
- **Stage 2 is a model judgment**. It runs only if Stage 1 passes, and it catches failures the checklist has no rule for. It returns one of three verdicts: pass; incomplete, with a reminder that names the URLs already read and the tools not yet called; or a multi-topic query where the answer covered only some topics.

A rejection does not end the run. For an incomplete answer, or a Stage 1 failure, the reminder is added to the conversation and control returns to the loop. For a multi-topic query, the class label is cleared and the query goes back to the classifier. If the judge's reply cannot be parsed, the answer passes, so a broken judge call cannot cause endless retries.

There is no retry limit, because an earlier fixed limit sometimes stopped a run that would have succeeded. The only stop is a ceiling of 20 turns, so that a query the system cannot answer still ends.

This is a simple version of the self-critique in Self-RAG (Asai et al., 2024). Self-RAG trains the model to emit reflection tokens and scores several signals for each generated segment. Mine is one pass/fail gate per attempt, and it needs no training.

3.6 Example: one query from start to finish

Take 如何辦理休學. This is the typical path from the design. The exact tool sequence varies between runs.

1. The classifier labels it a procedure question, so it takes the knowledge path.
2. For a leave-of-absence question the subdomain is normally `aca`, the academic affairs site. The model restates its goal and calls `search_texts`, which finds the leave-of-absence regulation page.
3. The page mentions form QP-T01-03-02. The form-ID pattern matches the tool result, and the resource lookup fetches the form. It reads the signature table and records that the registrar's office has several approval layers.
4. The form text names the offices on it. The office-name pattern matches, and the contact lookup fetches the contact details of each office named.
5. The loop gets one more turn. The model may read further pages, for example the refund rules.
6. Synthesis writes the answer: the stops in order, each office with its contacts, and the form. Stage 1 checks the step format and the approval layers. Stage 2 checks completeness. If either objects, the run goes back as described in Section 3.5.

4. Data

I crawled 153 NCCU subdomains and fetched 226,795 pages and documents: 196,750 HTML pages and 30,045 documents such as PDFs, about 35.5 GB of raw files. These figures come from the README of my crawler project (NCCU-Crawler). The raw files are not in this repository. After text extraction there are 203,510 page records (about 892 MB), one JSON line per page. Supplementary form content adds 414 more lines, for 203,924 in total. I re-counted the extracted files in `output/` and both numbers match. Extraction depends on file type:

- **HTML:** the main content area is parsed and links are kept as Markdown. Contact details in page footers are pulled out before the footer is dropped.
- **PDF:** a layout-aware extractor turns detected tables into Markdown. Image-only PDFs (scanned files with no text layer) are not extracted yet.

A separate index maps every form ID to its download URL and metadata. It backs the `get_form` tool. The supplementary content comes from 59 subdomains.

5. Evaluation setup

Models. The system runs on Gemma4:31b (Gemma Team, 2026), an open-weight model, without fine-tuning. In every experiment in this report I called the model through Ollama's cloud service (`gemma4:31b-cloud`), not on my own hardware, which does not currently have enough compute for a 31B model. The results therefore show what the architecture does with this model. They do not show that a local machine can serve it at a usable speed. As a reference, I also used Claude Sonnet 4.6 (Anthropic, 2026), acting as a general-purpose coding agent with access to the same corpus, not

through my pipeline. This is not a fair benchmark between systems of comparable cost, deployability, or architecture. It shows how much headroom the problem leaves for a capable model that is free of my design choices.

5.1 Queries

I tested 11 queries: six basic queries (Table 1) and five that each isolate a different way offices work together (Table 2).

Table 1. Basic queries.

Type	Query (topic)	Criteria	What it checks
Procedure	leave of absence	13	Every office and stop in the nine-stop chain
Procedure variant	resuming study after leave	9	A second procedure, including not adding an approval step that the first has and this one lacks
Knowledge	credit limit per semester	5	One fact, no multi-office structure
Resource	where to download the leave form	3	The correct form is linked directly
Contact	cashier's office phone	2	Correct extension and office name (not a person's name, which changes over time)
Compound	leave procedure plus library phone	15	Both halves stay complete

Table 2. Cross-office patterns.

Pattern	Query (topic)	Criteria	What it checks
Parallel	approvals for a department transfer	6	Two offices approve independently
Sequential handoff	dorm deposit refund	6	The second office needs the first office's signed output
Three-stage chain	double major / minor	6	Two departments then the registrar, in order
Conditional	exchange-credit exemption	7	The reviewing office depends on course type
Cross-institution	double major across a university consortium	7	Picks the right partner institution among similar names

I stopped at five patterns. The other candidate forms I read (freshman credit transfer, general-education credit recognition, military status survey) turned out to repeat a pattern already covered, to be a simpler version of one, or not to involve several offices at all.

5.2 Scoring

Each checklist item is scored 0, 1, or 2. Two points means the answer contains at least a minimum number of the item's required keywords. One point means it has some. Zero means none.

Checklists for Table 1 were made from Claude Sonnet 4.6 answers. Sonnet answered each query directly, acting as a general coding agent with access to the same corpus. I checked each answer against the source regulations and forms, then turned it into checklist items. Checklists for Table 2 were made from the source forms themselves.

What this means for Sonnet as an "upper bound". Because Table 1's checklists come from Sonnet's checked answers, Sonnet scores full marks on them by construction. That says the checklists are achievable. It does not measure Sonnet independently. The repo does record one real score: Sonnet 4.6 running as a full agent (`rag/proto1_direct.py`, mode 3) got 26/26 on the leave-of-absence procedure (`CLAUDE.md`, eval table). For Table 2 I did not run a scored Sonnet baseline.

5.3 How the scoring can go wrong

- **Items with no keywords pass automatically.** A few items (for example "does not invent an approval step", or "does not answer a form request with a full procedure") have no keywords. The script gives them full marks, on the assumption that I check them by hand. I did check them, and Table 3 reflects that. Running the script alone overstates the result. In one run it did: an answer that added a dean's-office approval step the form does not have still passed.
- **Keywords that are too narrow.** A correct answer can lose points because it uses a synonym or writes a number in Arabic digits instead of Chinese ones. This happened in two Table 2 checklists.
- **Keywords that collide.** A keyword for one fact can match text about another. In the exchange-credit checklist, the credit-limit keyword matched an unrelated deadline phrase with the same fraction. The answer passed an item it should have failed.

I fixed all three problems in the checklists and report each correction next to its result. Three of the five Table 2 checklists were changed once during evaluation.

5.4 Size of the evaluation

Each query is one example with one hand-built checklist. I ran most queries once. The leave-of-absence procedure query was run at least three times (24/26, 25/26, and 25/26 after the fix in Section 7.2), and the cross-institution query was run twice before its fix. This shows whether the system handles each targeted case. It does not measure how often it does so, and I did not measure sampling variance.

Each query is matched to its checklist by exact query text, not by the class my own classifier assigns. This keeps a routing error separate from an answer error.

6. Results

6.1 Basic queries

Table 3. Basic queries on Gemma4:31b, checked by hand.

Type	Score	Notes
Procedure	24/26	Refund-percentage table missing. One cashier's extension differs between two pages of the corpus. Another run of the same query scored 25/26.

Type	Score	Notes
Procedure variant	16/18	Adds a dean's-office approval step that the form does not have. The script gave 18/18.
Knowledge	8/10	Content is complete. The two lost points come from the keyword check: the answer gives the undergraduate floor inside a range ("12 to 25 credits") and the check expected it as a separate phrase.
Resource	2/6	Script 4/6. The answer names the right form but links the academic affairs page, not the download host, and it is a full procedure. The script passes the shape item automatically. See below.
Contact	4/4	Reports both extensions found in the corpus.
Compound	25/30	See Section 6.3.

Procedure. Every structural requirement is met: the nine stops across seven offices, the conditional stops, the three-level approval, the contact for each office, the pickup method, and the mental-health note. The refund table is on a linked page the agent did not open within its tool calls. The cashier's office has two different extensions on two pages of the corpus. The checklist expects one, and the answer gave the other.

Resource. 休學申請表在哪裡下載 ("where do I download the leave-of-absence form") should get a short answer with a direct link. I ran it once more for this report. The classifier routed it straight to the resource lookup, and the run finished in one loop turn. The answer named the right form (QP-T01-03-02), but it linked the academic affairs page instead of the form's download host (`mol.tke.nccu.edu.tw`), so the script gave 4/6. The answer was also a full seven-step procedure with every office's contact details. The checklist has an item for this: the answer must not be a full-procedure explanation. That item has no keywords, so the script passes it automatically. By hand it fails, and the score is 2/6. Earlier runs of this query differed. One, recorded on 2026-09-13, scored 6/6 by script with a correct link and a full procedure. Another took eight turns. So the problem is not only routing. When the query is routed correctly, the resource lookup fetches the form, the contact lookup adds every office named on it, and synthesis writes all of it out. I did not test why synthesis prefers a full procedure here.

Contact. The answer reports both cashier extensions instead of choosing one. I count this as correct because the corpus itself is inconsistent.

6.2 Cross-office patterns

Table 4. Cross-office patterns on Gemma4:31b, checked by hand.

Pattern	Score	Notes
Parallel	12/12	
Sequential handoff	12/12	First run was 9/12. The checklist rejected a job-title synonym and an Arabic numeral. I checked the answer against the form, found it correct, and widened the keywords.
Three-stage chain	10/12	Missing: an approved outside major automatically cancels the existing one. See below.

Pattern	Score	Notes
Conditional	12/14	The system routes each course type to the right office. It never states the credit-limit rule. The script first gave 14/14 because of the keyword collision.
Cross-institution	3/14, then 9/14 after a fix	See Section 7.2.

Reliability differs by pattern. Parallel and sequential cases reached the maximum. The three-stage chain and the conditional case each miss one specific fact. Cross-institution was the only case where the system answered about the wrong institution.

The missing rule in the three-stage chain. I could not tell whether the form text was never retrieved or was retrieved and dropped during synthesis. The trace I used prints only tool calls and their results. It skips the message type that the resource lookup uses to put form text into the conversation. The number of turns suggests the form was retrieved, which would make this a synthesis omission, but the trace cannot confirm it.

6.3 Compound query

Before I added the split step, one synthesis call handled both halves. The library half was fine. The leave half fell from 25/26 (asked alone) to 14/26. Whole categories vanished: the form to download and four of the seven office stops. The needed pages had reached the synthesis step, so retrieval was not the cause. One call answering two topics simply produced a worse answer for one of them.

With the split, the leave half now scores about the same as when asked alone. One smaller loss remains: the mental-health note is only partly there. It mentions a pregnancy-related condition but not the mental-health center or its phone number.

The library half has a different error. The answer gives a phone extension for 圖書資訊與檔案學研究所 (the Graduate Institute of Library, Information and Archival Studies) instead of 圖書館 (the library). The two names share words, and the retrieved pages did not clearly separate them. The office-name check from Section 3.3 did not run here. It had been built and tested for queries sent directly to the contact lookup. It does not run on sub-queries that find an office name through the normal loop.

7. What I changed and what happened

I made four changes. Two replaced a model decision with a step in code, one removed a model decision, and one cleaned a model's input in code. I list what I saw for each. I did not test them as a general rule.

7.1 Four changes

Change	Before	After
Follow forms by pattern (Section 3.3)	The model often mentioned a form ID in its goal but did not call <code>get_form</code> . The answer lacked the form content.	A fixed pattern on tool output triggers the fetch. The failure no longer appeared in my runs. No counts.

Change	Before	After
Detect office names with a string check first	The model was asked which office a short query names. On the bare query 出納組電話幾號 it returned nothing in 14 of 15 calls (1 of 15 succeeded), even though 出納組 appears word for word in the query and in exactly one entry of the office list. The same model found the office reliably when given a full page of text.	Before the model call, scan the query for any of the roughly 433 known office names. The scan's hits and the model's answer are combined. The model call still always runs, because the scan cannot handle informal names (住宿組 for 住宿輔導組). Result: 10 of 10 calls succeeded. Both figures are repeated calls on the same single query , not a set of different test cases, and the two samples differ in size (15 and 10).
Remove the pre-fetch relevance filter	The resource step judged which of about 32 candidate forms to fetch. I narrowed this to the one or two forms whose IDs appeared in the text. A sample of three to five runs looked better.	On a matched comparison of eight trials each, judging over all 32 succeeded 8 of 8, and the narrowed version 0 of 8. An earlier stable version had no filter: it fetched every form whose ID appeared in the text and let synthesis decide what to write. I went back to that. A wrong filter there costs only a cluttered answer. In the fetch step, it silently loses data. The 32-form judgment is kept for one case only: a direct resource request such as 休學申請表在哪裡下載, where the query contains no form ID to detect.
Strip links before the relevance judge sees a page	The judge saw the first 200 characters of each candidate page, including the anchor text of Markdown links.	I removed every Markdown link, text and URL, from that excerpt. Results are in Section 7.2.

The third row removes a step, and the fourth cleans the model's input. Neither replaces a model call with code.

7.2 The cross-institution failure

The query asked about a double major across a four-university consortium. The first run scored 3 of 14. The system picked a page about an unrelated agreement between NCCU and an arts university and described it in detail. The contacts, deadlines, and procedure it gave were real, so nothing was invented, but the page answered a different question. I ran the query a second time. The answer was again about the wrong institution, and this time it also stated that one institution was subordinate to the other, which is false. Two runs with the same error suggest a systematic cause, though two runs are a small sample.

I traced the cause to the relevance judge. The candidate page began with a link whose anchor text named the consortium. The page itself was about the arts university. The judge saw only the excerpt and could not tell a link label from the page's own content.

My first fix added a rule to the judge's prompt: check that the institution in the query matches the page's own content and not a link. I rejected this fix, because it only works if the model applies a new rule correctly every time, and it is written for one type of query. I stripped the links instead. This works on any page with the same problem. The query then scored 9 of 14. It found the consortium and its two-level approval structure. The remaining misses are the member universities' names and the enrollment quota. I re-ran the leave-of-absence query to check for regressions. It scored 25/26. Earlier runs of that query scored 24/26 and 25/26, so the fix did not change the result. Each score is one run.

8. Limitations

1. **Router classification.** The query classifier does not always classify correctly, and a misclassified query goes down a slower path: it enters the full multi-turn knowledge loop instead of the direct resource or contact lookup. The keyword rules were written around specific query shapes, such as an office's contact details, and can miss a related shape, for example a role-to-person lookup ("who is the dean"). When no rule gives a clear result, or two classes tie, a zero-shot model call decides, and I have not measured how accurate it is. The form-download query in Section 6.1 is not an example of this gap: in my latest run it was routed correctly. The office-name check (Section 3.3) has a related gap: it runs only on queries sent straight to the contact lookup, not on sub-queries that find an office name in the normal loop. That is why one compound sub-query returned the library institute instead of the library (Section 6.3). **Next step:** train a lightweight classifier (TF-IDF features and an SVM) to replace the zero-shot model classification. When its confidence is high, use its label directly. When it is low, pass the query to the model with updated category definitions, as a second opinion. This is in the spirit of Adaptive-RAG (Jeong et al., 2024), with the model fallback added. Training labels could come from which path each earlier query's self-evaluation actually passed. Running the office-name check on every path is a smaller step.
2. **Keyword search retries.** When FTS5 finds nothing, the agent falls back on `grep_texts` and has to retry with other keywords over many turns. The cause is that keyword search and substring search fail the same way when a query and the page that answers it use different words for the same thing: 休學 (leave of absence) and 保留學籍 (retaining student status) are related procedures with no shared substring. Each retry costs a tool call and a model turn, so a query that should take two or three turns can take many more. **Next step:** replace SQLite FTS5 with a dedicated BM25 engine, such as Lucene, Pyserini, or the Lemur Project. This would not close the vocabulary gap by itself, but it would make each retry cheaper. To reduce how often retries are needed, the agent could first generate a short hypothetical answer and expand the keyword query with its words, a light version of HyDE (Gao et al., 2023). That needs no change to the index, because it expands the keyword query and builds no embeddings.
3. **Scoring.** Some checklist items have no keywords and need a human check. Keywords can be too narrow or can collide with other text. I fixed the cases I found, but I cannot rule out more.

9. Related work

Retrieval without chunks. LongRAG (Jiang et al., 2024) reads long units instead of short chunks. ELITE (Wang et al., 2025) narrows a corpus by iterative model reasoning over raw text, with no embeddings. Subramanian et al. (2026) report that agentic keyword search reaches over 90% of the performance of vector-based RAG. My system uses no embeddings and reads whole pages, as they do. The difference is that the model chooses individual tool calls in an ordinary loop.

Loop techniques. I combined chain-of-thought (Wei et al., 2022), ReAct (Yao et al., 2023), and Plan-and-Solve (Wang et al., 2023). I did not test each one separately, so I cannot say which contributes most.

Routing and self-evaluation. Adaptive-RAG (Jeong et al., 2024) trains a classifier to route queries by complexity. Self-RAG (Asai et al., 2024) fine-tunes the model to critique itself. Both need training. My keyword-rule router and checklist gate need none and can be edited when a new failure appears. I did

not compare their accuracy to those methods.

Small models and harness design. Zhou et al. (2026) show that letting a model choose its own strategy fails on weaker models. Zhang (2026) argues that the code around a model deserves deliberate design. My four changes in Section 7 point in the same direction. I reached them by testing my own system, and they are not a test of that argument.

10. Conclusion

A keyword-search agent on an open-weight model that can be deployed locally can answer cross-office questions from a university's own documents. The agent combines chain-of-thought prompting, ReAct, Plan-and-Solve, query routing, and self-evaluation in one loop, with no fine-tuning. In my tests it handled most required content, including whole signature chains, with sources. It was weakest where the model had to make a judgment that no code check backed up: it answered a form-download question with a full procedure instead of a short answer, it confused two offices with similar names, and it omitted some rules. When I replaced such judgments with checks in code, removed them, or cleaned their input, the results improved in each case I tested. Since the evaluation is small and single-run, these are observations from one system, not general results.

References

- Anthropic. System Card: Claude Sonnet 4.6. 2026. <https://www.anthropic.com/document/claude-sonnet-4-6-system-card>
- Asai, A., Wu, Z., Wang, Y., Sil, A., Hajishirzi, H. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *ICLR 2024*. arXiv:2310.11511.
- Chan, C.-M., et al. RQ-RAG: Learning to Refine Queries for Retrieval Augmented Generation. arXiv:2404.00610, 2024.
- Gao, L., Ma, X., Lin, J., Callan, J. Precise Zero-Shot Dense Retrieval without Relevance Labels. *ACL 2023*. arXiv:2212.10496.
- Gemma Team. Gemma 4 Technical Report. arXiv:2607.02770, 2026.
- Jeong, S., et al. Adaptive-RAG. *NAACL 2024*. arXiv:2403.14403.
- Jiang, Z., Ma, X., Chen, W. LongRAG. arXiv:2406.15319, 2024.
- Ma, X., et al. Query Rewriting for Retrieval-Augmented Large Language Models. *EMNLP 2023*. arXiv:2305.14283.
- Robertson, S. E., et al. Okapi at TREC-3. *TREC-3*, NIST, 1994.
- Subramanian, S., Akinfaderin, A., Zhang, Y., Singh, I., Khanuja, M., Singh, S., Ladeira Tanke, M. Keyword search is all you need: Achieving RAG-Level Performance without vector databases using agentic tool use. arXiv:2602.23368.
- Wang, L., et al. Plan-and-Solve Prompting. *ACL 2023*. arXiv:2305.04091.
- Wang, Z., et al. ELITE: Embedding-Less Retrieval with Iterative Text Exploration. arXiv:2505.11908, 2025.

- Wei, J., et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *NeurIPS 2022*. arXiv:2201.11903.
- Yao, S., et al. ReAct. *ICLR 2023*. arXiv:2210.03629.
- Zhang, T. Harness as an Asset: Enforcing Determinism via the Convergent AI Agent Framework (CAAF). arXiv:2604.17025, 2026.
- Zhou, H., Tan, Z., Zhang, Z., et al. Select-then-Solve: Paradigm Routing as Inference-Time Optimization for LLM Agents. arXiv:2604.06753, 2026.